Document Title

Author Name

August 7, 2026

Contents

1 Introduction

The utility of CompCert is that it provides a platform for reasoning formally about all the potential behaviours of compiled source-code. Given a piece of C source-code, the user must be able to infer (and reason about) all possible runtime-behaviours that could be produced upon executing the target program generated by compiling it. CompCertTSO is the first and to our knowledge only existing extension of CompCert that provides a formal model for reasoning about the behaviours of compiled source-code in a concurrent setting. Our intent is to extend CompCertTSO to accurately model RC11 concurrent behaviours, not TSO.

2 Design

In order to properly model this problem in Rocq, we must first be explicit about what we mean by program behaviour. As in CompCertTSO, we define the behaviour of a program to be the set of all possible observable behaviours that can be produced by executing the program. More precisely, we define observable behaviours to be traces of events that are produced by executing the program. Events are defined to be,

event ::= call fn_id args | return type value | exit code | fail

Notice that we do not include memory events in our definition of observable behaviours. Between compilation passes memory events may be introduced, removed or reordered. Therefore, we require a model of program behaviour that is independent of the precise memory traces that produce it. In this concurrent context, traces of events are generated by the interleaving of thread-local traces produced by multiple threads. As in CompCertTSO we decouple thread-local semantics, and therefore thread-local traces, from the memory model, allowing potentially non-sensical behaviours at the thread local level. We assert only at the whole system level that when a memory event is produced by a thread-local trace, it must be consistent with the memory model. This filters out all the non-sensical behaviours at the thread-local level in order to compose a consistent whole-system behaviour. Thread-local traces are defined by the following events,

thread-local-event ::= external event | mem mevent | τ | start tid p | exit

Memory traces are defined by the following events.

memory-event ::= read loc type value | write loc type value | rmw loc type value | fence | malloc loc type | free

Whereas in CompCertTSO, the behaviour of the memory model was defined by the TSO abstract machine, an operational model that uses ordered threadwise buffers to simulate the total store order semantics, we express the behaviour of the RC11 memory model as a set of axioms that decide upon the consistency of an execution graph generated by incrementally adding events and relations that satisfy these axioms. A memory event (node) and its corresponding primary edges (po, mo, rf) are added to the execution graph via a legal step in the RC11 memory model semantics. A legal step is defined as a step that does not violate any of the axioms of the RC11 memory model. We rely on the work of IMM to define the consistency of the RC11 execution graphs. IMM defines events to be reads, writes or fences, we must also account for the addition of malloc and free events. To do this we interpret malloc and free events as writes to the location they allocate or free. Initially all locations are considered to be unallocated. Malloc events allocate a location with an undefined value. Free, on the other hand, deallocates a location and therefore removes it from the set of allocated locations (writes the unallocated value to the location). Importantly our operational semantics does not allow malloc to allocate

the same location twice, ie. calls to malloc always return an event with a fresh location. This is an important constraint that is not enforced by the RC11 memory model, but is necessary to ensure that our operational semantics are well-defined. Allowing malloc to allocate the same location twice, raises the question of when can an event be considered unallocated and hence available to be re-allocated. This question is beyond the scope of this work, but we note that it is an important question that must be answered in order to define a realistic operational semantics for malloc and free.

The small step semantics of the RC11 memory model is defined by the following set of inference rules, that define the conditions under which a memory event can be added to an execution graph. Execution graphs are denoted using Ψ, δ denotes the newly added event and Ψ' denotes the new execution graph after the addition of δ. Functions over execution graphs are denoted using the subscript Ψ, functions that are independent of the execution graph are denoted without a subscript. The isAllocated_Ψ predicate is defined to be true if the value of the terminal write event to the location of the newly added event is not the unallocated value.

$$\text{isAllocated}_\Psi \text{ x } := \text{val}(\text{last}_\Psi(x)) \neq \text{VUnalloc}$$

Its negation, $\text{isNotAllocated}_\Psi$, is defined to be true if the only event on that location is the initial unallocated write. We note that it should come as a lemma that it is decidable whether a location is allocated or not, since the execution graph is finite and the initial write is always present.

$$\text{isNotAllocated}_\Psi \text{ x } := \nexists \sigma \in E_\Psi \text{ s.t. } (W_{init} \text{ x } ; mo; \sigma) \notin V_\Psi$$

$$\frac{\begin{array}{c}\text{RC11Consistent}(\Psi) \\ \text{RC11Consistent}(\Psi')\end{array} \qquad \begin{array}{c}E_{\Psi'} = E_\Psi \cup \{\delta\} \\ \delta = \text{R x type v} \\ v \neq \text{VUnalloc}\end{array}}{\Psi \xrightarrow{\text{MRead x type v}} \Psi'} \text{MRead}$$

$$\frac{\begin{array}{c}\text{RC11Consistent}(\Psi) \\ \text{RC11Consistent}(\Psi')\end{array} \qquad \begin{array}{c}E_{\Psi'} = E_\Psi \cup \{\delta\} \\ \delta = \text{W x type v} \\ \text{isAllocated}_\Psi\ \text{x}\end{array}}{\Psi \xrightarrow{\text{MWrite x type v}} \Psi'} \text{MWrite}$$

$$\frac{\begin{array}{c}\text{RC11Consistent}(\Psi) \\ \text{RC11Consistent}(\Psi')\end{array} \qquad \begin{array}{c}E_{\Psi'} = E_\Psi \cup \{\delta\} \\ \delta = \text{R x type VUnalloc} \\ \text{isNotAllocated}_\Psi\ \text{x}\end{array}}{\Psi \xrightarrow{\text{MReadFail x}} \Psi'} \text{MReadFail}$$

$$\frac{\begin{array}{c}\text{RC11Consistent}(\Psi) \\ \text{RC11Consistent}(\Psi')\end{array} \qquad \begin{array}{c}E_{\Psi'} = E_\Psi \cup \{\delta\} \\ \delta = \text{W x type v} \\ \text{isNotAllocated}_\Psi\ \text{x}\end{array}}{\Psi \xrightarrow{\text{MWriteFail x}} \Psi'} \text{MWriteFail}$$

$$\frac{\begin{array}{c}\text{RC11Consistent}(\Psi) \\ \text{RC11Consistent}(\Psi')\end{array} \qquad \begin{array}{c}E_{\Psi'} = E_\Psi \cup \{\delta\} \\ \delta = \text{W x type VUndef} \\ \text{isNotAllocated}_\Psi\ \text{x}\end{array}}{\Psi \xrightarrow{\text{MMalloc x}} \Psi'} \text{MMalloc}$$

$$\frac{\begin{array}{c}\text{RC11Consistent}(\Psi) \\ \text{RC11Consistent}(\Psi')\end{array} \qquad \begin{array}{c}E_{\Psi'} = E_\Psi \cup \{\delta\} \\ \delta = \text{W x type VUnalloc} \\ \text{isAllocated}_\Psi\ \text{x}\end{array}}{\Psi \xrightarrow{\text{MFree x}} \Psi'} \text{MFree}$$

For each of the rules that add a read event, the following predicate is satisfied as a consequence of RC11 consistency and the definition of $E_{\Psi'}$.

$$\exists \theta\, \gamma \text{ s.t. } \theta \in E_\Psi \wedge \gamma \in E_\Psi \wedge \gamma = \text{W x type v} \wedge V_{\Psi'} = V_\Psi \cup \{\theta; \text{po}; \delta, \gamma; \text{rf}; \delta\}$$

Similarly, for each rule that adds a write event, the following predicate is satisfied as a consequence of RC11 consistency and the definition of $E_{Psi'}$.

$$\exists \theta\, \gamma \text{ s.t. } \theta \in E_\Psi \wedge \gamma \in E_\Psi \wedge \text{is\_write}(\gamma) \wedge V_{\Psi'} = V_\Psi \cup \{\theta; \text{po}; \delta, \gamma; \text{mo}; \delta\}$$

The operational memory model LTS is then composed with the thread-local LTS to define the whole-system semantics of a program. The small-step semantics of the whole-system is defined by the following inference rules, that define the conditions under which a thread-local step can be added to a whole-system trace.

$$\frac{\Sigma \xrightarrow{\text{TERead x type v}} \Sigma' \qquad \Psi \xrightarrow{\text{MRead x type v}} \Psi'}{\Psi\,;\,\Sigma \xrightarrow{\tau} \Psi'\,;\,\Sigma'}\ \textsc{Read}$$

$$\frac{\Sigma \xrightarrow{\text{TEWrite x type v}} \Sigma' \qquad \Psi \xrightarrow{\text{MWrite x type v}} \Psi'}{\Psi\,;\,\Sigma \xrightarrow{\tau} \Psi'\,;\,\Sigma'}\ \textsc{Write}$$

$$\frac{\Sigma \xrightarrow{\text{TERead x type v}} \Sigma' \qquad \Psi \xrightarrow{\text{MReadFail x}} \Psi'}{\Psi\,;\,\Sigma \xrightarrow{\text{Fail}} \Psi'\,;\,\Sigma'}\ \textsc{ReadFail}$$

$$\frac{\Sigma \xrightarrow{\text{TEWrite x type v}} \Sigma' \qquad \Psi \xrightarrow{\text{MWriteFail x}} \Psi'}{\Psi\,;\,\Sigma \xrightarrow{\text{Fail}} \Psi'\,;\,\Sigma'}\ \textsc{WriteFail}$$

Traces generated by the whole-system semantics are then further filtered to produce the observable behaviours of a program. Specifically this ensures that all traces containing the fail event terminate at the point of failure and do not continue executing as is permitted by the whole-system semantics.

3 Results

Present your results here.

4 Conclusion

Summarize your conclusions here.